



Операционни системи - Учебни материали

Клас: 11б | Предмет: Операционни системи (ИУЧ - СПП)

1. Прекъсвания и хардуерна поддръжка

1.1. Какво е прекъсване (interrupt)

За да разберем как операционната система управлява хардуера, трябва да се запознаем с концепцията за **прекъсванията (interrupts)**.

Прекъсването е сигнал към процесора, че се е случило важно събитие, което трябва да се обработи веднага. Това е основен механизъм, чрез който хардуерните устройства комуникират с процесора.

Без прекъсванията процесорът ще трябва постоянно да проверява (да "пита") всяко устройство дали има нещо за него - това би било изключително неефективно. Прекъсванията позволяват на устройствата сами да сигнализируют когато има нужда от внимание.

Когато пристигне прекъсване, процесорът извършва следните стъпки:

1. Спира текущото изпълнение на програмата
2. Запазва текущото състояние (контекст)
3. Изпълнява **рутина за обработка на прекъсването (interrupt handler)**
4. Връща се към предишната програма и продължава от мястото, където е спрял

В същност, прекъсванията са **сигнали от хардуера (или софтуера) до процесора**, които уведомяват за важно събитие.

1.2. Видове прекъсвания

Прекъсванията могат да се класифицират по различни начини според техния произход и поведение:

- **Хардуерни прекъсвания** – генерирани от физически устройства като клавиатура (при натискане на клавиш), мишка, твърд диск (при завършване на операция), таймер и други периферни устройства.
- **Софтуерни прекъсвания** – предизвикани от самата програма. Класически пример е извикването на **system call** - когато програма иска услуга от операционната система (например отваряне на файл или заявка за памет).
- **Маскируеми и немаскируеми прекъсвания** – според възможността за временно игнориране. Маскируемите могат да бъдат временно блокирани от операционната система, докато немаскируемите (като критични системни грешки) трябва винаги да се обработват веднага.

1.3. Таймери и тяхната роля в хардуерната поддръжка

Таймерът е специален хардуерен компонент, който играе ключова роля в работата на операционната система.

Основна функция: Таймерът периодично изпраща прекъсване към процесора на определени интервали от време (например всеки няколко милисекунди).

Приложения:

- **Измерване на време** – ОС използва таймера за поддържане на точна системна дата и час, за измерване на продължителността на операции и за отчитане на времетраенето на различни процеси.
- **Планиране на процеси (time slicing)** – това е може би най-важната функция. Таймерът позволява на операционната система да разделя процесорното време справедливо между различните програми. Когато изтече определено време (например 100 милисекунди), таймерът изпраща прекъсване, което кара ОС да спре текущия процес и да даде шанс на друг процес да използва процесора. Така се постига **многозадачност** - усещането, че много програми работят едновременно, макар че процесорът в един момент изпълнява само една.
- **Синхронизация** – различни компоненти на системата могат да се координират въз основа на прекъсванията от таймера.

Без таймера операционната система няма да може да управлява ефективно процесите и многозадачността няма да бъде възможна.

2. Структура на ОС: ядро и обвивка

2.1. Ядро (kernel)

Ядрото (kernel) е основната част на операционната система. То управлява критичните ресурси на компютъра: **процесора, паметта, устройствата и файловата система.**

Ядрото работи в **привилегирован режим** (kernel mode), който му дава пълен достъп до хардуера. Това означава, че ядрото може директно да контролира процесора, паметта и всички устройства.

Важно е да разберем, че приложенията **не говорят директно с хардуера**. Вместо това, когато една програма иска да прочете файл или да изпрати данни по мрежата, тя прави заявка към ядрото, което след това изпълнява операцията от негово име.

В обобщение, ядрото е **основната част от ОС, която управлява ресурсите** и осигурява стабилна и сигурна работа на системата.

2.2. Обвивка (shell)

Обвивката (shell) е **интерфейс между потребителя и ядрото**. Тя е слойът, чрез който потребителят взаимодейства с операционната система.

Как работи обвивката:

- Приема команди от потребителя (текстови или графични)
- Изпраща ги към ядрото за обработка
- Получава резултатите от ядрото
- Показва резултатите на потребителя

Обвивката може да бъде:

- **Текстова (command-line interface)** – потребителят въвежда текстови команди
- **Графична (GUI shell)** – потребителят взаимодейства чрез прозорци, менюта и икони

Примери за текстови shell:

```
Windows: cmd.exe, PowerShell  
Linux/Unix: bash, zsh, sh
```

2.3. Системни функции (system calls)

Системните функции (system calls) са специални функции, чрез които приложенията искат услуги от ядрото. Те представляват **интерфейса между потребителските програми и ядрото**.

Когато една програма иска да извърши операция, която изисква привилегиран достъп (например работа с файлове, създаване на процеси, заявка за памет, мрежови операции), тя не може да го направи директно. Вместо това, програмата прави system call, който прехвърля контрола към ядрото.

Примери за system calls:

- Отваряне на файл
- Четене/писане на данни
- Създаване или прекратяване на процес
- Заявка за памет
- Изпращане на данни по мрежата

Основни функции на операционната система:

- Управление на процеси
- Управление на паметта
- Управление на файловата система
- Управление на устройства (I/O management)
- Защита и сигурност

2.4. Архитектури на ОС – монолитна и микроядро

Операционните системи могат да бъдат проектирани по различни архитектури. Двете основни подхода са **монолитна архитектура** и **микроядро архитектура**.

Монолитна архитектура

При монолитната архитектура всички основни услуги на ОС (драйвери, файлови системи, управление на процеси и др.) се изпълняват в **едно адресно пространство на ядрото**. Всички компоненти са част от едно голямо ядро.

Предимства:

- По-висока бързина – по-малко превключвания между потребителско и ядрено пространство
- По-проста комуникация между модулите – всички компоненти са в едно пространство

Недостатъци:

- Грешка в един модул може да сrine цялата система
- По-трудна поддръжка и разширяване

Микроядро архитектура

При микроядро архитектурата ядрото съдържа само **най-основни функции** (низко ниво комуникация, планиране, управление на паметта). Останалите услуги (драйвери, файлови системи и др.) работят като **отделни процеси в потребителско пространство**.

Предимства:

- По-добра надеждност – ако един модул се сrine, той не влияе на останалите
- По-добра сигурност – услугите са изолирани
- По-лесно обновяване на отделни услуги без да се рестартира цялата система

Недостатъци:

- По-голям брой превключвания между потребителско и ядрено пространство
- По-голям брой съобщения между процесите
- Като резултат – по-бавна работа в сравнение с монолитната архитектура

3. Видове ОС според поколенията

3.1. Еволюция на операционните системи

Операционните системи са преминали през няколко етапа в своето развитие, познати като поколения:

- **Първо поколение** – пакетни системи (batch processing). Програмите се изпълняват една след друга без човешка намеса.
- **Второ поколение** – системи с multiprogramming. Няколко програми се зареждат в паметта едновременно, но процесорът все още изпълнява само една в даден момент.
- **Трето поколение** – системи с разделяне на времето (time-sharing). Множество потребители могат да използват системата едновременно, а процесорът бързо превключва между задачите.
- **Четвърто поколение** – персонални компютри и мрежови ОС. Развитие на персоналните компютри и локални/глобални мрежи.

4. Команди в конзола (Windows и Linux)

4.1. Навигация и преглед на файлове

Работата с конзолата изисква познаване на основните команди за навигация и управление на файлове. Ето основните команди за работа с директории и файлове:

Действие	Windows	Linux / Unix
----------	---------	--------------

Показване на файлове и директории	<code>dir</code>	<code>ls</code>
Промяна на директория	<code>cd</code> име_на_директория	<code>cd</code> име_на_директория
Показване на текущата директория (пълния път)	<code>echo %cd%</code>	<code>pwd</code>
Създаване на директория	<code>mkdir име</code>	<code>mkdir име</code>

4.2. Копиране, местене и изтриване

Операциите за манипулиране на файлове са от съществено значение при работа с конзолата:

Действие	Windows	Linux / Unix
Копиране на файл	<code>copy source dest</code>	<code>cp source dest</code>
Местене / преименуване на файл	<code>move old new</code>	<code>mv old new</code>
Изтриване на файл	<code>del file</code>	<code>rm file</code>

Примери за употреба:

Копиране на файл

Windows: `copy C:\Documents\file.txt C:\Backup\file.txt`

Linux: `cp /home/user/document.txt /home/user/backup/docu`

Преименуване на файл

Windows: `move oldname.txt newname.txt`

Linux: `mv oldname.txt newname.txt`

```
# Изтриване на файл
Windows: del unwanted.txt
Linux: rm unwanted.txt
```

4.3. Помощ и документация

И двете операционни системи предоставят начини за получаване на помощ за командите:

- **Windows:** `help` – показва списък с команди или помощ за конкретна команда (напр. `help dir`)
- **Linux/Unix:** `man` команда – отваря "manual page" (ръководство) за дадена команда, което съдържа подробна документация за употребата ѝ

Тези команди са от съществено значение, когато не си спомняте точно как се използва дадена команда или искате да научите повече за нейните опции.

5. Пакетни системи и мениджъри

5.1. Основни пакетни мениджъри в Linux

Пакетните мениджъри са инструменти за инсталиране, обновяване и управление на софтуер в Linux дистрибуциите. Различните дистрибуции използват различни пакетни мениджъри:

Дистрибуция	Пакетен мениджър	Примерна команда за инсталиране
Debian / Ubuntu	<code>apt</code> (Advanced Package Tool)	<code>sudo apt install package</code>
Red Hat / Fedora / CentOS	<code>yum</code> или <code>dnf</code>	<code>sudo yum install package</code>

Arch Linux	pacman	sudo pacman -S package
------------	--------	---------------------------

Важно е да запомним съответствието между дистрибуцията и нейния пакетен мениджър:

- **apt** – използва се в Debian и Ubuntu дистрибуции
- **yum/dnf** – използва се в Red Hat, Fedora и CentOS дистрибуции
- **pacman** – използва се в Arch Linux дистрибуция

6. Защита на паметта и сигурност

6.1. Защо е нужна защита на паметта

Защитата на паметта е критичен механизъм в операционните системи, който осигурява стабилност и сигурност:

- **Изолиране на процесите** – предпазва операционната система и различните програми една от друга
- **Предотвратяване на несанкциониран достъп** – процесът не може да чете или пише произволна памет, която не му принадлежи
- **Намаляване на риска** – намалява риска от сризове, защото грешка в една програма не може да повреди друга или самата ОС
- **Защита срещу злоупотреби** – предотвратява злонамерени програми да достъпват или променят чувствителни данни

6.2. Основни механизми за защита и сигурност

Операционните системи използват множество механизми за осигуряване на сигурност:

Защита на паметта

Основните механизми за защита на паметта включват:

- **Базови и лимитни регистри** – определят границите на паметта, до която може да достъпва даден процес
- **Сегментиране** – разделяне на паметта на логически сегменти с различни права за достъп
- **Странициране (paging)** – разделяне на паметта на страници, които могат да се управляват независимо

Други механизми за защита

- **Контрол на достъпа** – управление на правата на потребители и групи за достъп до файлове и ресурси (read/write/execute права)
- **Криптиране** – шифриране на данни на диск и в мрежата за защита срещу неоторизиран достъп
- **Автентикация и авторизация** – удостоверяване на самоличността на потребителите чрез пароли, многофакторно удостоверяване и управление на роли
- **Антивирус софтуер** – откриване и премахване на злонамерен софтуер
- **Файървол** – филтриране на мрежовия трафик за защита срещу атаки
- **Журнали (logs)** – записване на системни събития за проследяване на активността и откриване на подозрителни дейности

7. Файлова структура и команди

7.1. Файлова система

Файловата система представлява **иерархична структура** от директории (папки) и файлове, която организира данните на компютъра.

Структура на файловата система

- **Коренова директория (root)** – най-горното ниво в йерархията:
 - В Linux/Unix: `/` (наклонена черта)
 - В Windows: `C:\` , `D:\` и т.н. (дискове)
- **Път до файл (path)** – комбинация от директории и име на файл, която указва точното местоположение на файла в йерархията

Примери за пътища:

Windows:

C:\Users\user\Documents\file.txt

C:\Program Files\Application\app.exe

Linux/Unix:

/home/user/document.txt

/usr/bin/program

/var/log/system.log

Файловата система позволява логична организация на данните, което улеснява тяхното намиране и управление.

Успех в изучаването на операционните системи! 🎓